DEPLOYING NODE.JS APPLICATION ON ELASTIC BEANSTALK UING CI/CD PIPELINE

PROJECT IMPLEMENTATION

Project Development

The TravelGo website was developed using Node.js and Express.js in Visual Studio Code. The application was designed to provide a simple interface where users can explore travel destinations and book trips online. The project files, including the application code, package files, Dockerfile, Procfile, and static resources, were organized in a structured manner for easy development and deployment.

Creating the Node.js Application

A new Node.js project was initialized using npm. The Express.js framework was installed to create the web server. The main application logic was written in the app.js file, which handles incoming HTTP requests and returns the TravelGo web pages to users.

Project Structure

The project consists of the following files and folders:

app.js – Main application file

public/ – Stores CSS, images, and JavaScript files

data/ – Stores travel package information

package.json – Contains project metadata and dependencies

package-lock.json – Locks dependency versions

Dockerfile – Defines Docker image creation steps

Procfile – Specifies the startup command for Elastic Beanstalk

buildspec.yml – Defines build instructions for AWS CodeBuild

This structure keeps the project organized and easy to maintain.

Docker Implementation

A Dockerfile was created to containerize the application. The Dockerfile uses the official Node.js image, installs project dependencies, copies the application files, exposes the required port, and starts the application.

Containerization ensures that the application runs consistently across different environments without dependency conflicts.

Creating the AWS Elastic Beanstalk Application

An application was created in AWS Elastic Beanstalk by selecting the Node.js platform. The project ZIP file was uploaded, and Elastic Beanstalk automatically created the required cloud resources, including the EC2 instance and application environment.

The deployment process was monitored until the environment status changed to Ready and the health status became Green.

CI/CD Pipeline Implementation

A Continuous Integration and Continuous Deployment (CI/CD) pipeline was created using AWS CodePipeline.

The pipeline included the following stages:

Source Stage – Retrieves the latest project from GitHub.

Build Stage – Uses AWS CodeBuild to build the application based on the buildspec.yml file.

Deploy Stage – Deploys the latest application version to AWS Elastic Beanstalk.

This automation reduces manual deployment effort and ensures that application updates are deployed efficiently.

Application Deployment

After the deployment process was completed successfully, AWS Elastic Beanstalk generated a public domain URL. The application was accessed through this URL to verify that all features were functioning correctly.

The deployment confirmed that the application was successfully hosted on the AWS cloud platform.

Testing

The deployed TravelGo website was tested by accessing the generated AWS domain URL.

The following functionalities were verified:

Website loads successfully.

Home page is displayed correctly.

Travel destinations are visible.

Trip booking interface is accessible.

Navigation works correctly.

Responsive layout is displayed properly.

The application performed successfully during testing.

Result

The TravelGo application was successfully developed, containerized using Docker, integrated with GitHub, deployed through AWS CodePipeline and AWS Elastic Beanstalk, and made accessible through a public web URL. The implementation provided practical experience in web application development, cloud deployment, Docker containerization, and CI/CD automation.